

Guide d'Architecture Backend

Système GMAO : Workflow & Sécurité

Document Pédagogique et Technique

Ce document détaille l'implémentation de la sécurité JWT, le cycle de vie d'une intervention (Machine à états) et l'algorithme transactionnel de gestion des stocks.

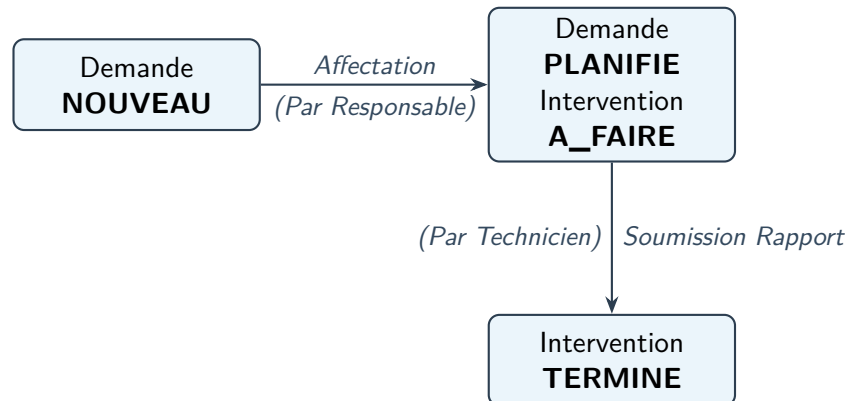
Contents

1	Le Workflow Métier : Cycle de vie d'une Demande	2
1.1	Schéma du Cycle de Vie (State Machine)	2
1.2	Les 3 Étapes du Workflow	2
2	L'Algorithme de Gestion des Stocks	2
3	La Sécurité Avancée : JSON Web Token (JWT)	3
3.1	Schéma du Flux d'Authentification	3
3.2	L'Implémentation pas à pas	3

1 Le Workflow Métier : Cycle de vie d'une Demande

Le cœur de la GMAO réside dans l'orchestration des tâches entre les différents acteurs. Nous avons implémenté une **Machine à états** stricte pour garantir la cohérence des données.

1.1 Schéma du Cycle de Vie (State Machine)



1.2 Les 3 Étapes du Workflow

1. **Création (Demandeur)** : Un employé signale une panne. Le système crée une Demande avec le statut NOUVEAU.
2. **Affectation (Responsable)** : Via le InterventionService, le responsable sélectionne un Technicien. Le système lie les entités, change la Demande en PLANIFIE et initialise l'Intervention en A_FAIRE.
3. **Clôture (Technicien)** : Après réparation, le technicien soumet un Rapport. Le système valide le rapport, met à jour les stocks, et clôture l'Intervention en TERMINE.

2 L'Algorithme de Gestion des Stocks

Lors de l'étape de clôture, le système doit gérer la consommation des pièces de rechange. C'est l'opération la plus sensible, car elle modifie l'inventaire physique.

Concept : La Transaction @Transactional

La méthode `soumettreRapport` est annotée `@Transactional`. Cela signifie que si une erreur se produit (ex: pièce en rupture de stock), **toutes** les opérations précédentes s'annulent. Le rapport n'est pas sauvegardé, l'intervention ne se clôture pas. C'est le principe du "Tout ou Rien" pour éviter les corruptions de base de données.

Algorithme de Déduction du Stock (RapportService)

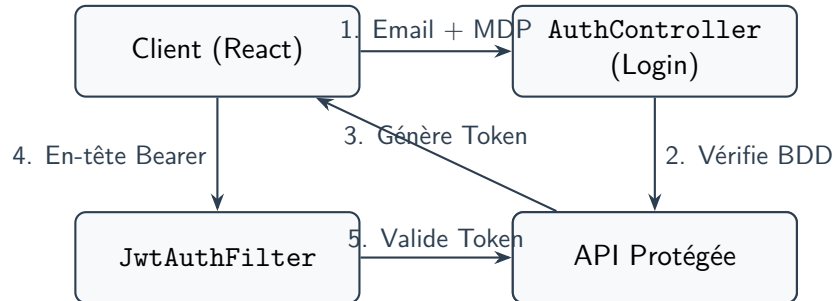
```

// ... pour chaque ID de pièce reçu du Frontend : PieceRechange piece = pieceRepository.findById(idPiece).orElseThrow();
if (piece.getQuantiteEnStock() > 0) { // 1. Déduction mathématique
    piece.setQuantiteEnStock(piece.getQuantiteEnStock() - 1);
    // 2. Sauvegarde de la nouvelle quantité pieceRepository.save(piece);
    // 3. Ajout au "Panier" du rapport piecesUtilisees.add(piece); } else { // RUPTURE DE STOCK :
    Annule toute la transaction ! throw new RuntimeException("Stock insuffisant !"); }
  
```

3 La Sécurité Avancée : JSON Web Token (JWT)

Pour sécuriser notre API REST, nous avons implémenté le standard JWT (Stateless).

3.1 Schéma du Flux d'Authentification



3.2 L'Implémentation pas à pas

Concept : 1. L'Usine à Tokens : JwtService.java

C'est le moteur cryptographique. Il contient notre clé secrète en Base64. Sa méthode `generateToken()` assemble l'email, l'expiration (24h) et signe le tout.

Concept : 2. Le Garde du Corps : JwtAuthenticationFilter.java

Cette classe intercepte chaque requête. Elle extrait le mot Bearer, lit le Token, vérifie la signature, et si elle est valide, inscrit l'utilisateur dans le contexte de sécurité de Spring.

Configuration et Verrouillage (SecurityConfig.java)

```

http .sessionManagement(session -> session.sessionCreationPolicy(STATELESS)) .authorize-
HttpRequests(auth -> auth .requestMatchers("/api/auth/**").permitAll() // Login Public .anyRe-
quest().authenticated() // Tout le reste privé ) .addFilterBefore(jwtAuthFilter, UsernamePasswor-
dAuthenticationFilter.class);
  
```

Bilan Architectural : L'API Backend est désormais capable de gérer des processus métiers complexes, des transactions sécurisées sur la base de données, tout en maintenant un standard de sécurité de haut niveau.